Guia técnico do projeto

Este guia explica, com linguagem direta, como as partes principais da aplicação se conectam. Ele complementa o README e serve como ponto de partida para quem precisa entender ou apresentar o código.

Versão para leitura/compartilhamento: guia técnico detalhado em PDF.

1. O que a aplicação faz

O projeto é um site Django para a May Artes 3D. Ele tem dois fluxos de planilhas:

1. **Planilhas do chatbot:** o usuário escolhe um tema, responde às perguntas, salva os registros no SQLite e pode baixar um arquivo Excel.
2. **Planilhas externas:** o usuário envia um arquivo `.xlsx`, escolhe uma aba, altera células existentes e baixa uma cópia editada. O original não é sobrescrito nem guardado no histórico.

A aplicação não se conecta diretamente ao Google Sheets, OneDrive ou outros serviços em nuvem.

2. Como as pastas se organizam

```
gerador_planilhas/
    settings.py       Configurações do Django, do banco e dos arquivos estáticos.
    urls.py           Rotas gerais: painel administrativo e aplicativo home.

home/
    models.py         Estrutura dos dados guardados no banco.
    views.py          Regras, validações, APIs e criação/edição dos arquivos Excel.
    urls.py           Endereços da página e das APIs.
    templates/
        home.html     Estrutura HTML da página.
    migrations/       Criação das tabelas e dados iniciais do roteiro.
    tests.py          Testes automáticos dos fluxos do aplicativo.

templates/static/home/
    css/home.css      Aparência e comportamento responsivo.
    js/home.js        Interações do chatbot e do editor Excel no navegador.

db.sqlite3            Banco de dados local usado durante o desenvolvimento.
manage.py             Atalho para executar comandos do Django.
requirements.txt      Bibliotecas necessárias para instalar o projeto.
```

3. O caminho de uma requisição

Quando o navegador solicita uma página ou envia dados, o fluxo geral é:

```
Navegador → home/urls.py → função em home/views.py → SQLite ou arquivo Excel → resposta ao navegador
```

- **URL:** escolhe qual função deve atender a solicitação.
- **View:** recebe os dados, valida o pedido e coordena as operações.
- **Model:** representa os dados salvos no banco.
- **Template e JavaScript:** exibem a página e atualizam a interface.
- **Resposta:** pode ser uma página HTML, dados JSON ou um arquivo para download.

O Django usa o token **CSRF** nas requisições que enviam dados. Isso ajuda a impedir que outro site envie gravações sem autorização do navegador.

4. Dados do chatbot

Os modelos em `home/models.py` descrevem as tabelas:

- **TemaBot:** um assunto do chatbot, como pedidos, vendas, produtos, estoque ou eventos.
- **PerguntaBot:** o campo, o tipo, o texto, a obrigatoriedade e as opções de uma pergunta. As perguntas pertencem a um tema e têm uma ordem.
- **MensagemBot:** textos que o chatbot apresenta durante a conversa.
- **Planilha:** uma planilha criada pelo usuário para um tema.
- **RegistroConversa:** respostas de uma conversa, vinculadas a um tema e, opcionalmente, a uma planilha.
- **RegistroOperacional:** registros antigos que também podem aparecer no histórico.

As perguntas e mensagens ficam no SQLite, não fixas no JavaScript. Por isso, o roteiro pode ser consultado no Django Admin sem mudar o código da interface.

5. Como o chatbot cria e edita registros

1. Ao abrir a página, `home()` em `home/views.py` monta o histórico e entrega ao template os endereços das APIs.
2. `home.js` consulta a rota `api/roteiro/` e recebe os temas, perguntas e mensagens em JSON.
3. O navegador apresenta as perguntas e mantém as respostas da conversa temporariamente.
4. Ao confirmar, o JavaScript envia as respostas à API de criação ou edição.
5. A view verifica tema, campos, tamanho, opções e tipos (texto, número ou data).
6. Se os dados forem válidos, o Django grava ou atualiza os registros no SQLite.
7. O navegador atualiza a tela e mostra uma opção para baixar a planilha.

Para criar uma planilha nomeada, a view salva a planilha e o primeiro registro dentro de uma transação. Assim, os dois são gravados juntos ou nenhum deles é gravado se houver falha.

Rotas principais do chatbot

Endereço	Método	Para que serve
`/`	GET	Mostra a página inicial e o histórico recente.
`/api/roteiro/`	GET	Entrega temas, perguntas e mensagens do chatbot.
`/api/planilhas/`	GET	Lista as planilhas criadas.
`/api/planilhas/criar/`	POST	Cria uma planilha e seu primeiro registro.
`/api/planilhas/<id>/`	GET	Consulta perguntas e registros da planilha.
`/api/planilhas/<id>/registros/`	POST	Adiciona um registro à planilha.
`/api/planilhas/<id>/registros/<registro_id>/`	PATCH	Atualiza um registro existente.
`/planilhas/arquivo/<id>/baixar/`	GET	Baixa uma planilha criada pelo chatbot.

Os endereços são definidos em `home/urls.py`; os nomes das rotas permitem ao Django criar URLs sem deixá-las espalhadas pelo código.

6. Como o Excel do chatbot é gerado

As funções de exportação em `home/views.py` usam a biblioteca **openpyxl**:

1. Criam uma pasta de trabalho e uma aba.
2. Escrevem os cabeçalhos e registros.
3. Convertem números e datas para tipos reconhecidos pelo Excel.
4. Aplicam formatação básica, como cabeçalho colorido, filtro e largura de colunas.
5. Criam o arquivo na memória e retornam uma resposta HTTP com `Content-Disposition: attachment`, indicando ao navegador que deve baixá-lo.

O arquivo não precisa ser salvo em uma pasta do servidor para ser baixado.

No chatbot, o botão de download é um link direto para essa resposta do servidor. O navegador cuida do início e da gravação do arquivo; a interface não busca o Excel com `fetch` nem cria um link temporário.

7. Como funciona o editor de planilhas externas

No navegador

O editor fica na seção **Edite um arquivo Excel** em `home/templates/home.html`.

1. `home.js` lê o arquivo escolhido pelo usuário.
2. O navegador envia o arquivo e o token CSRF à rota de edição, usando `FormData`.
3. O servidor responde com os nomes das abas e as células da aba selecionada.
4. O JavaScript cria uma tabela de campos editáveis.
5. A interface guarda apenas as células que realmente foram alteradas.
6. Ao clicar em **Baixar planilha editada**, o JavaScript coloca as alterações num campo oculto e envia o formulário HTML normal com o arquivo original e o token CSRF.
7. O formulário envia a resposta para um `iframe` oculto. Assim, a página e a planilha que está sendo editada não são substituídas pela resposta do servidor.
8. O servidor aplica as alterações e devolve uma nova cópia `.xlsx` com `Content-Disposition: attachment`; o navegador inicia o download.

Esse envio nativo é intencional: alguns navegadores bloqueiam downloads iniciados depois de uma chamada assíncrona de `fetch`. Usando um formulário submetido diretamente pelo clique, o navegador consegue tratar a resposta como download. O `iframe` usa a política `SAMEORIGIN`, permitindo à página exibir erros JSON do servidor sem abrir os arquivos Excel dentro da área da página.

No servidor

`editar_planilha_externa()` em `home/views.py` coordena duas etapas:

- **Pré-visualização:** abre o arquivo, seleciona uma aba e devolve células em JSON.
- **Download:** recebe alterações, valida cada posição e valor, aplica as mudanças e envia o Excel resultante como anexo para download.

`abrir_planilha_externa()` verifica o formato `.xlsx`, o tamanho e a aba selecionada. Também limita o conteúdo descompactado e o número de células para evitar processar arquivos excessivamente grandes.

Ao editar, a aplicação tenta preservar o tipo original da célula para números, datas e valores booleanos. Um texto digitado que começa com `=` é mantido como texto, em vez de ser convertido numa fórmula.

Limites atuais do editor externo

- Aceita arquivos `.xlsx` de até **2 MB**.
- Permite editar uma aba com até **5.000 células** por vez.
- Altera valores em células existentes; não oferece comandos para inserir ou excluir linhas, colunas ou abas.
- Preserva as outras abas e a formatação que a biblioteca consegue ler e gravar.
- Fórmulas existentes são mantidas, mas o editor não calcula seus resultados.
- O arquivo é processado para a solicitação e não é adicionado ao banco nem ao histórico.
- O download é uma cópia; para substituir o arquivo original, o usuário precisa fazê-lo fora do site.

8. Para que servem os arquivos da interface

- `home.html`: define as seções, formulários, mensagens de status e pontos onde o JavaScript monta conteúdo.
- `home.js`: conversa com as APIs usando `fetch`, cria controles e mostra erros ou resultados sem recarregar a página. Para transferir arquivos, usa links ou o formulário HTML nativo para deixar o navegador iniciar o download.
- `home.css`: define cores, espaços, cartões, tabela de células e ajustes para telas menores.
- `settings.py`: informa ao Django quais aplicativos carregar, onde está o SQLite e onde procurar arquivos estáticos.

O navegador recebe a página HTML e os arquivos CSS/JavaScript. As operações de validação e de manipulação do Excel são executadas pelo servidor Python.

9. Como executar e testar

No PowerShell, na pasta do projeto:

```
python -m venv venv
.\venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python manage.py migrate
python manage.py runserver
```

Abra `http://127.0.0.1:8000/`. Para rodar os testes automatizados:

```
python manage.py test home
```

Os testes de `home/tests.py` verificam a validação do chatbot, o salvamento e edição de registros, a exportação do Excel e a prévia/edição de planilhas externas.

10. Termos técnicos em poucas palavras

- **API:** endereço usado pelo navegador para pedir ou enviar dados ao servidor.
- **JSON:** formato de texto estruturado usado para transportar dados entre JavaScript e Django.
- **View:** função Django que processa uma requisição e monta uma resposta.
- **Model:** definição de uma tabela e dos dados que ela guarda.
- **Migração:** instrução versionada que cria ou altera tabelas no banco.
- **CSRF:** proteção usada pelo Django para validar requisições que alteram dados.
- **Arquivo em memória:** arquivo temporário mantido durante o processamento, sem gravá-lo numa pasta permanente.
- **Teste automatizado:** código que simula solicitações e confere se o resultado esperado foi produzido.

---

11. Diagrama das tabelas e relacionamentos

O diagrama representa os modelos Django atuais. `1:N` significa “um para muitos”: um tema pode ter várias perguntas, por exemplo.



Como ler os relacionamentos

- `PerguntaBot.tema_id` aponta para `TemaBot`: cada pergunta pertence a um tema.
- `Planilha.tema_id` aponta para `TemaBot`: cada planilha tem um tema; o tema não pode ser removido enquanto houver planilhas (`PROTECT`).
- `RegistroConversa.tema_id` aponta para `TemaBot`.

- `RegistroConversa.planilha_id` pode ficar vazio para registros antigos/avulsos. Quando preenchido, aponta para uma planilha. Se a planilha for apagada, os registros vinculados também são apagados (`CASCADE`).
- `MensagemBot` guarda textos por chave, sem chave estrangeira para um tema.
- `RegistroOperacional` e `usuarios` não têm relações com as tabelas acima no modelo atual.

Os campos `respostas` de `RegistroConversa` e `opcoes` de `PerguntaBot` são JSON. Isso permite guardar conjuntos de respostas/opções sem criar uma coluna SQL separada para cada resposta.

Observação sobre `usuarios`: o modelo existe, mas não participa do fluxo atual de login. O campo `senha` é texto comum no modelo; não deve ser usado para autenticar pessoas reais sem migrar para o sistema de usuários e hash de senhas fornecido pelo Django.

12. De onde vêm as perguntas e respostas

Perguntas e textos exibidos

As perguntas **não são escritas diretamente no JavaScript**. As definições persistentes ficam no banco:

1. `home/migrations/0003_mensagembot_temabot_and_more.py` cria os temas e suas perguntas iniciais. A lista `temas` tem o código do campo, o título da coluna Excel, o texto da pergunta, o tipo e as opções.
2. A mesma migração cria mensagens de interface, como boas-vindas, confirmação e erros, na tabela `MensagemBot`.
3. Depois da migração, Django Admin pode editar esses dados no banco.
4. `roteiro_chatbot()` em `home/views.py:67-89` consulta temas ativos, perguntas relacionadas e mensagens e devolve JSON.
5. `initializeChatbot()` em `home.js:530-543` baixa esse JSON. A variável `script` mantém o roteiro no navegador durante a página.

Exemplo simplificado de uma pergunta que está no banco:

```
tema: venda
campo: quantidade
cabeçalho: Quantidade
pergunta: Qual é a quantidade vendida?
tipo: number
obrigatória: sim
```

Cada atributo tem um uso:

- `campo` é a chave que será usada no objeto de respostas e na coluna lógica.
- `cabecalho` é o nome legível mostrado no formulário e no Excel.
- `pergunta` é a frase apresentada pelo chatbot.
- `tipo` determina o controle de formulário e a validação.
- `obrigatoria` decide se um valor vazio pode ser enviado.
- `opcoes` transforma o campo em lista de opções e restringe os valores válidos.
- `ordem` determina em qual sequência a pergunta aparece.

Como o chatbot guarda respostas enquanto pergunta

No navegador, `answers` começa como um objeto vazio. Cada resposta é guardada pela chave `campo`, por exemplo:

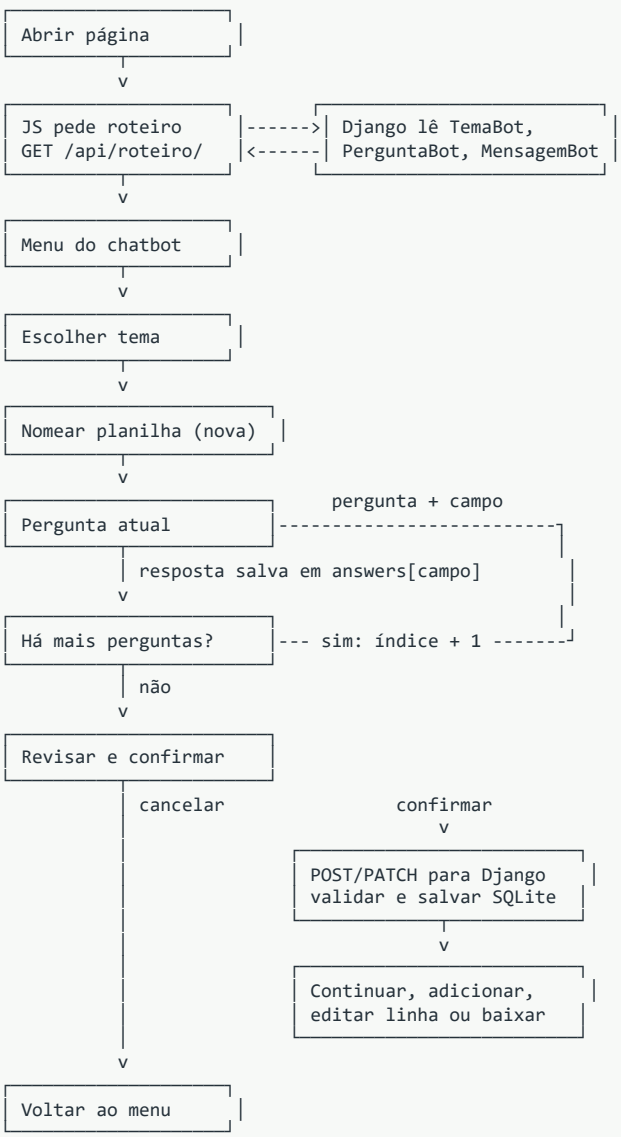
```
{
  "produto": "Miniatura",
  "quantidade": "2",
  "valor_total": "45.50",
  "data_venda": "2026-11-04"
}
```

O código pergunta uma coisa por vez:

1. `questionIndex` começa em zero.
2. `askCurrentQuestion()` escolhe `currentSheet.perguntas[questionIndex]`.
3. `showQuestionInput()` cria um `<select>` quando há opções; caso contrário cria um `<input>` do tipo configurado.
4. No envio do formulário, o valor é limpo com `trim()` e guardado como `answers[question.campo]`.
5. `questionIndex` aumenta e a próxima pergunta é exibida.
6. Quando o índice passa da última pergunta, a conversa mostra uma revisão e pede confirmação.

Esses dados são temporários no navegador até a confirmação. O JavaScript não cria o arquivo Excel com as respostas: ele envia as respostas ao Django, que valida e persiste os dados.

13. Diagrama do fluxo de conversação



Para uma planilha existente, o fluxo entra pelo menu **Editar uma planilha existente**: o navegador lista planilhas, carrega detalhes e oferece adicionar um registro, editar um registro existente ou baixar a planilha.

14. Como as respostas viram uma planilha

Esta aplicação tem duas etapas que não devem ser confundidas:

1. **Guardar os dados:** `saveEntry()` envia um objeto JSON com `tema`, `nome` e/ou `respostas`. A view valida esses dados e cria ou atualiza linhas do banco SQLite.
2. **Gerar o Excel:** quando o usuário escolhe baixar, uma rota `GET` busca os registros do banco. `gerar_arquivo_excel()` usa `openpyxl` para transformar cada resposta em uma linha e devolve o `.xlsx`.

Exemplo simplificado do banco antes da exportação:

```
Planilha: "Vendas de novembro", tema venda
Registro 1: {"produto": "Miniatura", "quantidade": "2", ...}
Registro 2: {"produto": "Chaveiro", "quantidade": "5", ...}
```

`gerar_arquivo_excel()` usa as perguntas ordenadas do tema para montar os cabeçalhos. Para cada registro, percorre as mesmas perguntas e busca o valor em `registro.respostas[pergunta.campo]`. Assim, o cabeçalho “Quantidade” corresponde à chave `quantidade`, mesmo que a posição das respostas esteja num JSON.

O arquivo não é a base de dados: ele é uma exportação dos dados armazenados. Editar o Excel baixado fora do site não atualiza automaticamente o SQLite.

15. Edição de um registro existente

Um registro do chatbot pertence a uma planilha e contém um objeto `respostas`. Para alterá-lo:

1. `chooseExistingSheet()` lista as planilhas com uma chamada `GET`.
2. `loadExistingSheet(id)` pede a planilha, as perguntas do tema e os registros existentes.
3. `chooseRecordToEdit()` apresenta os registros e associa cada botão ao objeto selecionado.
4. `beginEntry("edit", record)` guarda o registro em `selectedRecord` e copia suas respostas para `answers`.
5. `showQuestionInput()` mostra uma pergunta de cada vez, mas preenche cada campo com a resposta antiga.
6. O usuário pode mudar as respostas ou mantê-las como estão.
7. `saveEntry()` identifica o modo `edit`, usa o método `PATCH` e envia os dados ao endereço que contém os identificadores da planilha e do registro.
8. `salvar_registro_planilha()` valida os dados outra vez no servidor, procura o registro **dentro da planilha selecionada**, substitui o JSON `respostas` e salva.
9. A resposta JSON contém o registro atualizado. O navegador atualiza seu estado e a exportação seguinte refletirá os valores novos.

Não é criada uma segunda linha ao editar: a chave primária do registro continua igual. O escopo `planilha=planilha` na busca do registro impede editar por essa rota um registro que pertence a outra planilha.

16. Explicação passo a passo dos trechos mais complexos

As referências abaixo usam números de linha dos arquivos atuais. O objetivo é acompanhar cada bloco que realiza uma regra importante, sem repetir todo o código.

16.1 Validar as respostas recebidas

Arquivo: `home/views.py:108-148`, função `validar_respostas()`.

1. **108-110 — obter as regras:** lê as perguntas cadastradas para o tema e forma o conjunto das chaves permitidas.
2. **111-112 — conferir o formato lógico:** se faltou ou sobrou uma chave, rejeita o objeto antes de examinar os valores.

3. **114 — preparar resultado limpo:** as respostas normalizadas vão para um novo dicionário; o código não altera o objeto original recebido.
4. **115-119 — percorrer os campos esperados:** cada resposta tem que ser texto; espaços externos são removidos.
5. **121-125 — regras de preenchimento:** rejeita vazio em campo obrigatório, mantém vazio opcional e impede texto acima de 300 caracteres.
6. **126-127 — lista permitida:** quando a pergunta tem `opcoes`, só aceita um valor que esteja nessa lista.
7. **129-137 — número:** `Decimal` verifica se o texto é número sem os erros de arredondamento típicos de ponto flutuante. Também rejeita infinitos, valores negativos e valores acima do limite.
8. **138-142 — data:** `strptime` verifica o formato `AAAA-MM-DD` e rejeita datas impossíveis.
9. **144 — guardar valor validado:** salva a versão limpa e, no caso de números, normalizada.
10. **146-148 — resultado da função:** devolve as respostas e nenhum erro. No caminho de falha, retorna mensagem de erro em vez do dicionário.

Essa validação no servidor é necessária mesmo existindo validação no formulário: requisições HTTP podem ser enviadas sem usar a interface.

16.2 Criar uma planilha com o primeiro registro

Arquivo: `home/views.py:250-280`, função `criar_planilha()`.

1. **250-253 — aceitar somente POST e ler JSON:** o decorador limita o método HTTP; `ler_json()` também verifica se o corpo é um objeto JSON válido.
2. **255-260 — separar e conferir dados:** extrai o tema, nome e respostas; rejeita tipos incorretos e nomes vazios ou acima de 100 caracteres.
3. **262-265 — localizar tema ativo:** não permite criar novos registros para um tema inexistente ou desativado.
4. **266-268 — validar respostas:** reutiliza as mesmas regras da seção anterior.
5. **270-276 — transação atômica:** cria primeiro a planilha e depois o registro inicial que aponta para ela. `transaction.atomic()` garante que ambas as gravações sejam confirmadas juntas.
6. **277-280 — responder:** atualiza os metadados e devolve JSON com os dados do registro, da planilha e a URL de download.

16.3 Adicionar ou editar em uma planilha

Arquivo: `home/views.py:282-329`, função `salvar_registro_planilha()`.

1. **282-286 — selecionar a planilha:** retorna 404 se o identificador não existe.
2. **287-296 — ler e validar:** o corpo precisa conter `respostas` e obedecer às perguntas do tema.
3. **298 — ramificar pelo método HTTP:** `POST` representa criar uma nova linha; `PATCH` representa substituir uma linha existente.
4. **299-312 — adicionar:** impede usar POST no endereço de edição e cria um novo `RegistroConversa` ligado à planilha.
5. **314-320 — conferir o uso de PATCH:** impede atualizar a coleção inteira sem especificar um registro.
6. **322-326 — restringir o registro:** busca pelo ID do registro e pela planilha. Um ID válido de outra planilha não é suficiente.

7. **327-329 — atualizar:** substitui o JSON de respostas, salva apenas esse campo e monta a resposta comum de sucesso.

16.4 Guardar cada resposta e atualizar a pergunta

Arquivo: `templates/static/home/js/home.js:270-347`, funções `askCurrentQuestion()` e `showQuestionInput()`.

1. `askCurrentQuestion()` usa `questionIndex` para escolher uma pergunta de `currentSheet.perguntas`.
2. Se o índice não apontar para uma pergunta, todas já foram respondidas e `showReview()` é chamada.
3. `showQuestionInput(question)` cria um formulário e usa `question.campo` como nome/chave estável do campo.
4. Com `question.opcoes`, cria um menu `<select>` com as opções do banco; sem opções, cria um `<input>`.
5. `question.tipo` seleciona o tipo de entrada. `required`, `maxLength`, `min` e `max` ajudam o navegador a orientar o usuário.
6. Se o fluxo está editando um registro, `answers[question.campo]` fornece o valor anterior para preencher o campo.
7. Ao enviar, `preventDefault()` impede que o navegador recarregue a página.
8. Campo obrigatório vazio mostra erro e não avança o índice.
9. Caso contrário, grava `answers[question.campo]`, mostra a resposta no histórico e aumenta `questionIndex`.
10. A função chama novamente `askCurrentQuestion()` para continuar o ciclo.

16.5 Decidir entre criar, adicionar e editar no navegador

Arquivo: `templates/static/home/js/home.js:361-409`, função `saveEntry()`.

1. Começa com a URL de criação e método `POST`; `data` sempre recebe as respostas coletadas.
2. Modo `new` também envia o código do tema e nome da planilha.
3. Modo `add` troca a URL para a coleção de registros daquela planilha, mantendo `POST`.
4. Modo `edit` aponta para o ID do registro específico e troca o método para `PATCH`.
5. `requestJson()` serializa o objeto em JSON, envia CSRF e transforma respostas de erro HTTP em exceções.
6. Depois do sucesso, a resposta do servidor substitui o registro antigo no estado local quando o modo era edição; em criação/adição, o novo registro é acrescentado.
7. A tela mostra as próximas ações sem recarregar a página.
8. O bloco `catch` mantém os dados em memória e mostra o erro para que o usuário possa tentar de novo ou cancelar.

16.6 Abrir e validar o Excel externo

Arquivo: `home/views.py:372-401`, função `abrir_planilha_externa()`.

1. Procura o upload no campo multipart `arquivo`; rejeita envio sem arquivo.
2. Verifica a extensão `.xlsx` e o limite de arquivo comprimido de 2 MB.
3. Lê o conteúdo para memória, sem gravá-lo numa pasta.
4. `.xlsx` é um pacote ZIP. A função confere o número de itens e o tamanho descompactado antes de abrir o pacote com `openpyxl`.

5. Se o arquivo não for um pacote Excel válido, responde com erro HTTP 400 legível.
6. Escolhe a aba solicitada ou, inicialmente, a aba ativa.
7. Confirma que a aba existe e que sua grade estimada não excede 5.000 células.
8. Devolve a pasta de trabalho e a aba para a função de prévia/edição.

Esses limites reduzem o uso acidental/excessivo de memória. A checagem de extensão não substitui a validação real do conteúdo: por isso o pacote também é aberto e verificado.

16.7 Preservar tipos ao editar células

Arquivo: `home/views.py:403-450`, funções `valor_excel_para_texto()` e `editar_valor_excel()`.

- Na prévia, células vazias viram texto vazio; datas e horas usam formato ISO, que pode ser exibido em um campo de texto e interpretado novamente.
- Na edição, texto vazio limpa a célula.
- Se a célula original era booleana, só `true/verdadeiro` ou `false/falso` voltam a ser booleanos.
- Se era número, `Decimal` tenta interpretar o novo valor. Inteiros continuam inteiros quando possível; outros números finitos são gravados como float.
- Se era data/hora, tenta interpretar o formato ISO correspondente.
- Se a conversão não for possível ou a célula era texto, conserva a entrada como texto.

16.8 Prévia e download do Excel externo

Arquivo: `home/views.py:452-518`, função `editar_planilha_externa()`.

1. O decorador `require_POST` restringe a API a POST. `xframe_options_sameorigin` permite que a resposta seja carregada no `iframe` oculto da própria página, necessário para detectar erros do formulário de download.
2. Em toda chamada, `abrir_planilha_externa()` verifica o upload e seleciona a aba.
3. Sem o campo `alteracoes`, percorre linhas e colunas e devolve um JSON para construir a prévia.
4. Com `alteracoes`, converte o campo JSON e valida a quantidade e o tipo de cada item.
5. Cada alteração tem linha, coluna e texto; os índices precisam estar dentro da aba e o texto respeita o limite de caracteres do Excel.
6. `sheet.cell()` obtém a célula correta e `editar_valor_excel()` conserva tipos básicos.
7. Texto é marcado explicitamente como texto. Assim, entradas novas com `=` não viram fórmulas.
8. `workbook.save(BytesIO())` gera a cópia na memória.
9. `FileResponse` envia o tipo MIME `.xlsx`, um nome seguro e `Content-Disposition: attachment`, instruindo o navegador a baixar o arquivo.

16.9 Por que o formulário e o iframe ajudam no download

Arquivos do chatbot já existem numa rota GET, portanto um link normal pode abri-la e o servidor responde com `Content-Disposition: attachment`.

O editor externo precisa enviar o arquivo original e as alterações por POST **e** receber um arquivo. Se fizer isso com `fetch`, o JavaScript recebe os bytes primeiro; criar um link temporário depois de uma operação assíncrona pode ser bloqueado por alguns navegadores como download não iniciado diretamente pelo usuário.

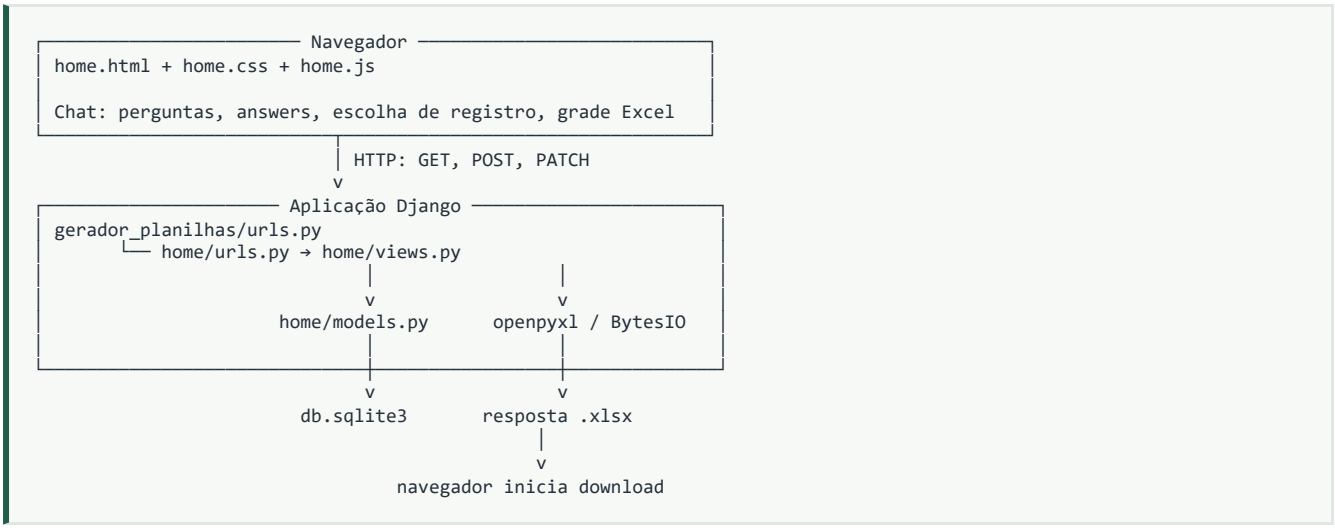
O código usa uma solução nativa:

1. O usuário clica no botão de download.
2. O JavaScript preenche o campo oculto `alteracoes`.
3. O formulário HTML envia o arquivo e o token CSRF como `multipart/form-data`.
4. O formulário tem `target="external-download-frame"`, então a resposta é direcionada ao iframe oculto, não à página principal.
5. Se a resposta for Excel, o cabeçalho `attachment` inicia o download no navegador. Se for JSON de erro, `showExternalDownloadError()` lê o texto do iframe e mostra a mensagem.

Arquivo: `templates/static/home/js/home.js:545-725`; formulário/iframe: `home/templates/home.html:113-139`.

17. Diagramas da arquitetura e das pastas

Componentes da aplicação



Árvore de pastas que participa do fluxo

```
gerador_planilhas/
├── manage.py
├── requirements.txt
├── db.sqlite3
├── gerador_planilhas/
│   ├── settings.py
│   ├── urls.py
│   ├── asgi.py
│   └── wsgi.py
├── home/
│   ├── models.py
│   ├── views.py
│   ├── urls.py
│   ├── tests.py
│   ├── migrations/
│   └── templates/home.html
├── templates/static/home/
│   ├── css/home.css
│   ├── js/home.js
│   └── img/
└── docs/
    ├── guia-tecnico.md
    └── guia-tecnico-detalhado.pdf
```

Responsabilidade de cada camada

Camada	Arquivos	Responsabilidade
Configuração	`gerador_planilhas/settings.py`	Aplicativos, banco de dados, templates, arquivos estáticos e fuso horário.
Rotas	`gerador_planilhas/urls.py`, `home/urls.py`	Relacionar um endereço HTTP a uma view.
Dados	`home/models.py`, `home/migrations/`	Descrever e criar tabelas e dados iniciais.
Regras	`home/views.py`	Validar requisições, acessar models, criar respostas JSON e gerar Excel.
Página	`home/templates/home.html`	Estrutura da página e campos da interface.
Interação	`templates/static/home/js/home.js`	Estado da conversa, pedidos HTTP e atualizações no DOM.
Apresentação	`templates/static/home/css/home.css`	Cores, dimensões, acessibilidade visual e responsividade.
Verificação	`home/tests.py`	Conferir respostas HTTP, gravações e conteúdo real dos Excel.

18. Como os testes conferem o comportamento

`home/tests.py` usa `TestCase` do Django. Cada teste pode criar um banco temporário, enviar solicitações pelo cliente de teste e conferir status, JSON e banco.

- O helper `create_sheet()` cria dados de exemplo através da mesma API usada pela aplicação.
- `external_workbook_file()` monta um `.xlsx` temporário com duas abas, formatação e uma fórmula; assim não depende de um arquivo de usuário.
- Os testes do chatbot verificam validação, criação, edição, isolamento entre planilhas e exportação de números/datas.
- Os testes do editor externo reabrem os bytes baixados com `openpyxl` e conferem valores, fórmulas mantidas, formatação, outras abas, cabeçalhos HTTP e rejeição de entradas inválidas.
- A execução é `python manage.py test home`.

19. Pontos importantes para manter o projeto

1. Ao mudar perguntas, confirme que `campo` continua sendo a mesma chave esperada pelo fluxo e que `cabecalho` é o texto desejado no Excel.
2. Ao mudar as validações de uma pergunta, mantenha frontend e backend coerentes. A validação do servidor é a autoridade final.
3. Ao mudar uma rota, use o nome da URL com `reverse()` no servidor e a configuração JSON entregue à página, evitando endereços duplicados.

4. Ao alterar a estrutura das tabelas, crie uma migração com `python manage.py makemigrations` e aplique-a com `python manage.py migrate`.
5. Ao alterar a geração do Excel, mantenha testes que abram o arquivo real e verifiquem os valores e tipos das células.
6. O modelo `usuarios` e `RegistroOperacional` são estruturas separadas do fluxo principal; confirme os usos antes de removê-los.
7. `settings.py` está configurado para desenvolvimento. Antes de publicar, revise `DEBUG`, `SECRET_KEY`, `ALLOWED_HOSTS`, banco, HTTPS e configurações de segurança.